



MOSAIC DIRECT 2.0 · SCHEDULING FASE 2

Scroll horizontal em tabelas largas

Decisão de UX e defesa técnica para o acesso a colunas que não cabem na tela, nas tabelas do Relatório de Protocolos (24 colunas) e Criar Protocolo (14 colunas).

Decisão: **visual contido (cabeçalho fixo) + botões de bloco < >**

PRODUTO

Mosaic Direct 2.0

TEMA

Navegação de tabelas largas

DATA

Junho de 2026

AUTOR

Victor Dias

CONTEXTO

O problema

As telas são **LWCs rodando dentro do Experience Cloud (LWR)** do Mosaic Direct, e o mesmo componente também roda na tela interna do CS (Lightning Experience). As tabelas têm **mais colunas do que cabe na largura da tela** — e, em tabelas largas, o usuário **frequentemente não percebe que há conteúdo à direita**. Esse é um problema clássico de descoberta em scroll horizontal (documentado pela Nielsen Norman Group).

Portanto, o critério nº 1 não é "rolar bonito": é **deixar óbvio que existem mais colunas e dar um controle fácil e acessível para chegar nelas** — funcionando igual no portal, no CS e no mobile.

COMO ORGANIZAR A DISCUSSÃO

São duas decisões independentes

Separar os dois eixos evita confundir "look" com "mecanismo" — e mostra que parte da solução já vem de graça.

DECISÃO A · LAYOUT VERTICAL

Tabela contida (cabeçalho fixo)

Confinar a tabela a um contêiner de altura fixa, com cabeçalho fixo e rolagem própria — o visual "uma tela só", sem scroll vertical sobrando. O time de engenharia confirmou que isso é **viável de forma nativa**, dentro do componente, sem travar a página.

DECISÃO B · MECANISMO HORIZONTAL

Como acessar as colunas escondidas

Botões de bloco < >, barra custom sempre visível, ou apenas o scroll nativo. Aqui pesam descoberta, precisão, acessibilidade e risco de implementação.

Ganho imediato: o visual de "tela travada" que buscávamos (cabeçalho fixo, sem scroll sobrando) é alcançável de forma nativa e segura — independente do mecanismo horizontal escolhido. Ele entra na decisão como base.

Mecanismo horizontal, critério a critério

Avaliação das três alternativas pelos critérios que mais afetam o usuário e o risco de entrega.

CRITÉRIO DE UX	BOTÕES ◀ ▶ (Opção 3)	BARRA CUSTOM (Opção 2)	SCROLL NATIVO PURO
Descoberta ("há mais à direita?")	Alta — botão é convite claro	Média — barra discreta	Baixa — no Mac some até rolar
Precisão (coluna específica)	Média — pula em blocos	Alta — arrasta a qualquer ponto	Alta
Eficiência (chegar longe)	Boa — 1-2 cliques previsíveis	Média — arrasto longo	Média
Acessibilidade (teclado / leitor de tela)	Alta — botão real, foco, ARIA	Baixa — thumb custom não substitui o nativo	Alta
Consistência (portal + CS + mobile)	Alta — igual em tudo	Média — scrollbar varia por navegador	Média
Carga visual	Média — 2 botões nas bordas	Média — barra permanente	Nenhuma
Risco de implementação	Baixo	Médio	Baixo

A **Opção 3 (botões)** domina a Opção 2 em quase tudo que importa — descoberta, acessibilidade e consistência — perdendo só em **precisão de arrasto**, ponto pequeno e mitigável (ver alavancas adiante).

POR QUE O LAYOUT CONTIDO IMPORTA

O maior ganho de UX vem do cabeçalho fixo

Independente do botão, confinar a tabela numa altura fixa entrega os benefícios de usabilidade mais altos — e é justamente o caminho nativo que a engenharia liberou.

- ✓ **Cabeçalho sempre visível** — "reconhecer em vez de lembrar" (heurística de Nielsen): o usuário nunca perde o nome da coluna ao rolar as linhas.
- ✓ **Ações/rodapé fixos** — sempre ao alcance, sem rolar a página inteira.

- ✓ **"Uma tela só"** – sem scroll vertical sobrando, leitura mais focada.
 - ✓ **Sem risco de plataforma** – entregue dentro do componente, sem mexer no Theme Layout / chrome da página.
-

DECISÃO RECOMENDADA

Visual contido (nativo) + botões de bloco ‹ ›

Adotar o **layout contido de altura fixa** (cabeçalho fixo, sem scroll sobrando) pelo caminho nativo, usando os **botões de bloco** ‹ › como mecanismo de scroll horizontal. UX e engenharia apontam para o mesmo lugar:

- ✓ Entrega o **visual que buscávamos** (look "tela travada") com baixo risco.
- ✓ Melhor **descoberta** e melhor **acessibilidade** (botão real, teclado, leitor de tela).
- ✓ **Consistente** em portal, CS e mobile; sem efeito colateral de plataforma.

PARA BLINDAR A DECISÃO

Três alavancas de UX (todas nativas)

Neutralizam o único ponto fraco dos botões (precisão) e elevam ainda mais a descoberta.

- ✓ **Congelar a 1ª coluna (Protocolo) e o cabeçalho** — padrão de Excel/Salesforce para tabelas largas: o usuário nunca perde a identidade da linha enquanto navega as colunas. Maior ganho de UX, barato e nativo.
- ✓ **Manter a rolagem nativa ativa por baixo dos botões** (trackpad/teclado) — os botões são a camada de descoberta; quem quer precisão usa o trackpad.
- ✓ **Sombra / "fade" na borda direita** indicando que há mais conteúdo — reforço de descoberta a custo zero.

ANTECIPANDO PERGUNTAS

Perguntas & respostas

"Botão não é menos preciso que arrastar uma barra?"

Sim, em precisão pura. Mas a barra custom tem custo de acessibilidade e QA (risco médio), e a precisão dos botões é resolvida congelando a 1ª coluna + mantendo o scroll nativo por baixo. O saldo de UX fica com os botões.

"Perdemos o visual 'tela travada' (Opção 1)?"

Não. O look é entregue pelo layout contido nativo — cabeçalho fixo, sem scroll sobrando — sem o risco de travar a página inteira que a engenharia apontou.

"Funciona igual no CS interno e no mobile?"

Sim — é o principal motivo de escolher os botões: tudo interno ao componente, sem depender do Theme Layout, com comportamento idêntico em portal, CS e mobile.